《打包解包技术》完整课堂讲义

目录

《打包解包技术》完整课堂讲义

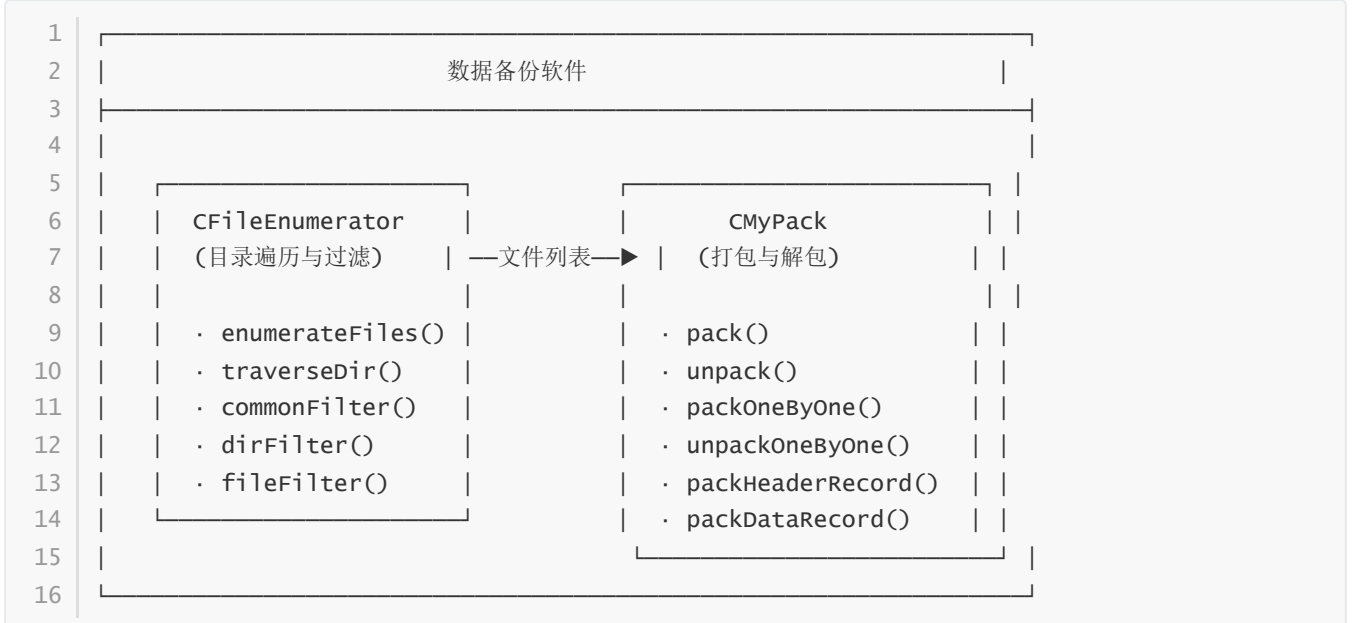
目录

1. 系统整体架构
    1.1 两者的协作流程
2. 核心数据结构与文件格式
    2.1 记录头结构体 `recordStruct`
    2.2 打包文件物理布局
    2.3 文件类型标识对照表
3. 文件遍历算法详解
    3.1 核心函数调用链
    3.2 `lstat()` 与 `stat()` 的关键区别
    3.3 路径处理技巧：`chdir` 的使用
4. 打包算法详解
    4.1 核心函数调用链
    4.2 头记录写入流程
    4.3 数据块写入流程
5. 解包算法详解
    5.1 核心函数调用链
    5.2 数据块读取逻辑
6. 🌳 完整案例走读：目录树打包解包全过程
    6.1 场景预设
    6.2 第一步：文件遍历（`CFileEnumerator`）
    6.3 第二步：打包（`CMyPack::pack`）
    6.4 第三步：解包（`CMyPack::unpack`）
    6.5 关键设计决策总结
7. 代码中的缺陷与改进方向
    7.1 ❌ 缺陷1：扩展头记录逻辑错误
    7.2 ❌ 缺陷2：目录时间戳恢复不完整
    7.3 ❌ 缺陷3：硬链接未处理
    7.4 ❌ 缺陷4：符号链接元数据未恢复
    7.5 ⚠️ 缺陷5：`realpath()` 可能绕过排除规则
8. 跨平台移植指南（Linux → Windows）
    Windows 下的四种链接机制
    Windows 下的路径分隔符处理
9. 课堂思考题
    Q1：遍历时 `readdir` 的顺序不确定，会影响打包/解包的正确性吗？
    Q2：如果源目录中包含指向自身的软链接（`ln -s . self`），遍历会死循环吗？
    Q3：为什么解包时要强制目标目录为空？
    Q4：打包后的文件，数据块为什么要固定 512 字节？为什么不用文件实际大小？
    Q5：如何修改代码以支持 Windows 下的符号链接？
    Q6：目录的 `mtime` 为什么不能准确还原？怎么修复？
10. 总结
    核心知识点
    代码设计亮点
    可改进方向

1. 系统整体架构

本系统包含两个核心类，职责分离，符合**单一职责原则**：

类名	职责	比喻
`CFileEnumerator`	遍历目录树，应用过滤规则，输出文件列表	📋 侦察兵
`CMyPack`	接收文件列表 → 打包成大文件 / 逆向解包	📦 搬运工



1.1 两者的协作流程



```
    |
    ▼
  打包文件 (.dbkp)
```

2. 核心数据结构与文件格式

2.1 记录头结构体 `recordStruct`

```c
struct recordStruct {
    // ====== 元数据字段 (二进制存储) ======
    char mode[sizeof(mode_t)];      // 文件类型 + 权限
    char uid[sizeof(uid_t)];        // 属主用户ID
    char gid[sizeof(gid_t)];        // 属组ID
    char mtime[sizeof(time_t)];     // 修改时间
    char atime[sizeof(time_t)];     // 访问时间
    char size[sizeof(off_t)];       // 文件大小 (仅普通文件)

    // ====== 设备号 (仅设备文件) ======
    char major[sizeof(unsigned int)];
    char minor[sizeof(unsigned int)];

    // ====== 路径长度 (解决二进制数据边界问题) ======
    char filePathLen[sizeof(int)];  // 文件名实际长度
    char linkPathLen[sizeof(int)];  // 链接目标实际长度

    // ====== 类型标识 ======
    char type;  // '-' 普通文件 | 'D' 目录 | 'L' 软链接 | 'B' 块设备
                // 'C' 字符设备 | 'F' 管道 | 'S' 套接字

    // ====== 路径数据区 (固定200字节) ======
    char filePath[200];   // 文件名或路径 (超长部分用扩展头)
    char linkPath[200];   // 软链接目标路径 (超长部分用扩展头)

    // ====== 数据负载区 (512字节) ======
    char block[512];      // 普通文件的文件数据
};
```

> **为什么用 `memcpy` 存二进制?** 因为 `mode_t`、`time_t` 等是整数类型，直接用二进制写入更紧凑，也无需解析（如Tar的八进制ASCII）。但这也意味着**不同架构/编译器可能不兼容**（如 `time_t` 大小在32位/64位系统上不同），这是一个可讨论的设计取舍。

2.2 打包文件物理布局

```
┌──────────────────────────────────────────────────────────┐
| commonHeaderStruct (公共头)                               |
|    · CRC64 (校验和)                                       |
|    · offset (数据起始偏移)                                |
|    · Algorithm (算法标识)                                 |
├──────────────────────────────────────────────────────────┤
| recordStruct (头记录) - 文件1                             |  ◄── 512字节
|    · type, mode, uid, gid, mtime, size, ...               |
```

```
|   · filePath = "file1.txt"                                          |
├─────────────────────────────────────────────────────────────────────┤
| [可选] 扩展头记录 × N                                                | ◄── 用于超长文件名/链接路径
├─────────────────────────────────────────────────────────────────────┤
| 数据块 1 (512字节)                                                   | ◄── 文件1的数据（不足512补零）
|   [ file1.txt 的有效数据 + 零填充 ]                                  |
├─────────────────────────────────────────────────────────────────────┤
| recordStruct (头记录) - 文件2                                        | ◄── 512字节
├─────────────────────────────────────────────────────────────────────┤
| 数据块 2 (512字节)                                                   | ◄── 文件2的数据
├─────────────────────────────────────────────────────────────────────┤
| ...                                                                 |
└─────────────────────────────────────────────────────────────────────┘
```

2.3 文件类型标识对照表

类型标识	宏定义	含义	恢复时使用的API
`-`	`S_ISREG()`	普通文件	`open()` + `write()`
`D`	`S_ISDIR()`	目录	`mkdir()`
`L`	`S_ISLNK()`	符号链接	`symlink()`
`B`	`S_ISBLK()`	块设备	`mknod()`
`C`	`S_ISCHR()`	字符设备	`mknod()`
`F`	`S_ISFIFO()`	命名管道	`mkfifo()`
`S`	`S_ISSOCK()`	套接字	（当前忽略）

3. 文件遍历算法详解

3.1 核心函数调用链

```
enumerateFiles(path, filter, result)
    |
    ├── getcwd(cwd)              // 保存当前工作目录
    ├── chdir(path)              // 切换到待遍历目录
    ├── traverseDir(".", filter, result)
    |       |
    |       ├── result.push_back(path)   // 先记录当前目录
    |       ├── opendir(path)
    |       ├── while ((entry = readdir(dir)) != NULL)
    |       |   |
    |       |   ├── 跳过 "." 和 ".."
    |       |   ├── 构造完整路径: path + "/" + fileName
    |       |   ├── commonFilter()  // 用户/组/时间过滤
    |       |   ├── if (entry->d_type == DT_DIR)
```

```
    |       |   |   ├── dirFilter()  // 路径排除过滤
    |       |   |   └── traverseDir(subPath)  // 递归!
    |       |   └── else
    |       |       ├── fileFilter()  // 大小/名字正则过滤
    |       |       └── result.push_back(filePath)
    |       └── closedir(dir)
    └── chdir(cwd)                // 恢复工作目录
```

3.2 `lstat()` 与 `stat()` 的关键区别

代码在 `traverseDir` 的过滤中使用了 `lstat()`，这是**正确且至关重要的。**

```cpp
// CFileEnumerator::commomFilter
struct stat buf;
if (lstat(path.c_str(), &buf) < 0) {  // 必须用 lstat!
    return true;
}
```

	`stat()`	`lstat()`
遇到软链接时	返回**目标文件**的信息	返回**链接本身**的信息
遍历安全	❌ 可能进入死循环	✅ 安全

错误示例：如果使用 `stat()`，遇到指向目录的软链接，会返回 `S_ISDIR` 为真，导致遍历器误入并死循环。

3.3 路径处理技巧：`chdir` 的使用

```cpp
// 保存当前工作目录
char cwd[PATH_MAX];
getcwd(cwd, sizeof(cwd));

// 切换到待遍历目录
chdir(path.c_str());

// 此时所有路径都是相对路径（如 "./subdir/file.txt"）
this->traverseDir(".", filter, result);

// 恢复工作目录
chdir(cwd);
```

好处：

1. 所有路径变为相对路径，打包时记录更短。
2. 解包时灵活性更高（可解压到任意目标目录）。
3. 避免绝对路径带来的安全问题。

4. 打包算法详解

4.1 核心函数调用链

```
pack(path, files, destFile)
    |
    ├── packCheck()              // 源路径存在性/可访问性检查
    ├── 创建 destFile，写入 commonHeaderStruct
    ├── getcwd(cwd) + chdir(path)  // 切换工作目录
    ├── for each file in files:
    |   └── packOneByOne(sf, file)
    |       ├── lstat(file, &buff)   // 获取元数据
    |       ├── packHeaderRecord()   // 写入头记录
    |       └── if (S_ISREG)         // 仅普通文件
    |           └── packDataRecord() // 写入数据块
    ├── chdir(cwd)               // 恢复工作目录
    └── updateFileCRC64()        // 更新校验和
```

4.2 头记录写入流程

```
int CMyPack::packHeaderRecord(Utils::CSimpleFile &sf,
                              const std::string &file,
                              struct stat &buff) {
    recordStruct record;
    memset(&record, 0, sizeof(record));

    // 1. 拷贝元数据
    memcpy(record.mode, &buff.st_mode, sizeof(buff.st_mode));
    memcpy(record.uid, &buff.st_uid, sizeof(buff.st_uid));
    memcpy(record.gid, &buff.st_gid, sizeof(buff.st_gid));
    memcpy(record.mtime, &buff.st_mtime, sizeof(buff.st_mtime));
    memcpy(record.atime, &buff.st_atime, sizeof(buff.st_atime));

    // 2. 根据文件类型填充
    if (S_ISREG(buff.st_mode)) {
        record.type = '-';
        memcpy(record.size, &buff.st_size, sizeof(buff.st_size));
    } else if (S_ISDIR(buff.st_mode)) {
        record.type = 'D';
    } else if (S_ISLNK(buff.st_mode)) {
        record.type = 'L';
        readlink(file.c_str(), linkPath, sizeof(linkPath));
        memcpy(record.linkPath, linkPath, ...);
    } else if (S_ISBLK(buff.st_mode)) {
        record.type = 'B';
        // 记录设备号
        unsigned int major = major(buff.st_dev);
        unsigned int minor = minor(buff.st_dev);
        memcpy(record.major, &major, sizeof(major));
        memcpy(record.minor, &minor, sizeof(minor));
    }
```

```
    // ... 其他类型

    // 3. 写入头记录 (固定512字节)
    sf.writeN(&record, sizeof(record));

    // 4. 如果文件名/链接路径超过200字节，写扩展头
    // ...
}
```

4.3 数据块写入流程

```
int CMyPack::packDataRecord(Utils::CSimpleFile &sf,
                            const std::string &file,
                            struct stat &buff) {
    recordStruct data;
    memset(data.block, 0, sizeof(data.block));

    CSimpleFile regularFile(file, O_RDONLY, -1);

    int ret = 0;
    while ((ret = regularFile.readN(data.block, sizeof(data.block))) > 0) {
        // 每次写入 512 字节块 (不足的已用 memset 补零)
        sf.writeN(data.block, sizeof(data.block));
        memset(data.block, 0, sizeof(data.block));
    }
    return 0;
}
```

关键点:

- 每次读取最多 512 字节，写入时固定 512 字节。
- 最后一次读取不足 512 字节时，由于 `memset` 已将 `data.block` 清零，写入的块中有效数据后的部分自动补零。
- 这保证了数据块**严格 512 字节对齐**，便于 `lseek` 跳过。

5. 解包算法详解

5.1 核心函数调用链

```
unpack(file, targetPath)
    |
    ├── unpackCheck()
    |   ├── 源文件存在性/可访问性检查
    |   ├── CRC64 校验
    |   ├── 公共头校验 (Algorithm 字段)
    |   └── ⚠ 目标目录必须为空! (遍历检查)
    ├── getcwd(cwd) + chdir(targetPath)
    ├── unpackOneByOne(sf)
    |   └── while (读取一个 recordStruct == 512字节)
    |       ├── 解析 type 字段
```

```
    |       ├── switch (type):
    |       |   ├── 'D' → mkdir()
    |       |   ├── '-' → open() + write() (根据 size 读数据)
    |       |   ├── 'L' → symlink()
    |       |   ├── 'B'/'C' → mknod()
    |       |   ├── 'F' → mkfifo()
    |       |   └── 'S' → 忽略
    |       ├── 恢复元数据 (chown + utime)
    |       └── 处理长文件名扩展头
    └── chdir(cwd)
```

5.2 数据块读取逻辑

```cpp
// 解包普通文件
if (type_ == '-') {
    CSimpleFile file(filePath_, O_CREAT | O_RDWR, mode_);

    int blk = size_ / sizeof(dataRecord);   // 完整块数
    int rest = size_ % sizeof(dataRecord);  // 最后一块的有效字节数

    // 读取完整块
    while (blk--) {
        sf.readN(&dataRecord, sizeof(dataRecord));
        file.writeN(dataRecord.block, sizeof(dataRecord));
    }

    // 读取最后一块（仅写有效字节数 rest）
    if (rest) {
        sf.readN(&dataRecord, sizeof(dataRecord));
        file.writeN(dataRecord.block, rest);  // 只写 rest 字节
    }
}
```

关键点：

- 数据块在打包时按 512 字节写入（含填充），解包时根据 `size` 只读取有效部分。
- 最后一块的填充数据被丢弃，恢复的文件大小精确为 `size_`。

6. 🌳 完整案例走读：目录树打包解包全过程

6.1 场景预设

假设源目录结构如下：

```
/home/user/test/
├── file1.txt          (普通文件, 100 字节)
├── file2.log          (普通文件, 50 字节)
└── subdir/            (子目录)
    ├── subfile1.txt   (普通文件, 200 字节)
    └── subfile2.doc   (普通文件, 300 字节)
```

6.2 第一步：文件遍历（`CFileEnumerator`）

调用：`enumerateFiles("/home/user/test", filter, result)`

① 切换工作目录

```
chdir("/home/user/test");   // 进入待遍历目录
```

② 递归遍历过程

第一层（根目录 `.`）：

- `result.push_back(".");` ← **先记录当前目录本身**
- 打开目录 `.`，`readdir` 读取条目（假设顺序：`file2.log` → `subdir` → `file1.txt`）

条目	类型	操作
`file2.log`	`DT_REG`	`result.push_back("file2.log")`
`subdir`	`DT_DIR`	**递归调用** `traverseDir("./subdir")`
`file1.txt`	`DT_REG`	`result.push_back("file1.txt")`

第二层（子目录 `./subdir`）：

- `result.push_back("./subdir");` ← **先记录子目录自身**
- 打开 `./subdir`，读取条目（假设顺序：`subfile2.doc` → `subfile1.txt`）

条目	类型	操作
`subfile2.doc`	`DT_REG`	`result.push_back("./subdir/subfile2.doc")`
`subfile1.txt`	`DT_REG`	`result.push_back("./subdir/subfile1.txt")`

③ 遍历结果（`result` 向量内容）

```
索引 0: "."
索引 1: "file2.log"
索引 2: "./subdir"                ← 父目录在子文件之前（先序保证）
索引 3: "./subdir/subfile2.doc"
索引 4: "./subdir/subfile1.txt"
索引 5: "file1.txt"
```

> **课堂重点**：
>
> - `"./subdir"` **一定出现在** `"./subdir/subfile2.doc"` **之前。**
> - `file1.txt` 和 `file2.log` 的顺序可能反过来，但不影响正确性（平级文件互不依赖）。
> - 这就是**先序遍历**：先记录目录自身，再递归子目录。

6.3 第二步：打包（`CMyPack::pack`）

调用：`pack("/home/user/test", result, "backup.dbkp")`

① 切换工作目录

```
chdir("/home/user/test");   // 与遍历时保持一致
```

② 依次处理每个文件

步骤	文件	`lstat` 结果	写入内容
1	`"."`	`S_ISDIR` ✅	**头记录** (type='D')，无数据块
2	`"file2.log"`	`S_ISREG` ✅，size=50	**头记录** (type='-') + **1个数据块**
3	`"./subdir"`	`S_ISDIR` ✅	**头记录** (type='D')，无数据块
4	`"./subdir/subfile2.doc"`	`S_ISREG` ✅，size=300	**头记录** (type='-') + **1个数据块**
5	`"./subdir/subfile1.txt"`	`S_ISREG` ✅，size=200	**头记录** (type='-') + **1个数据块**
6	`"file1.txt"`	`S_ISREG` ✅，size=100	**头记录** (type='-') + **1个数据块**

③ 打包文件物理布局

```
┌──────────────────────────────────────────────────────────────┐
| commonHeaderStruct                                           |
├──────────────────────────────────────────────────────────────┤
| [头记录1] 文件: "."              type='D'                    | 512B
├──────────────────────────────────────────────────────────────┤
| [头记录2] 文件: "file2.log"      type='-', size=50           | 512B
├──────────────────────────────────────────────────────────────┤
| [数据块1] file2.log (50B有效 + 462B \0填充)                   | 512B
├──────────────────────────────────────────────────────────────┤
| [头记录3] 文件: "./subdir"       type='D'                    | 512B
├──────────────────────────────────────────────────────────────┤
| [头记录4] 文件: "./subdir/subfile2.doc" type='-', size=300   | 512B
├──────────────────────────────────────────────────────────────┤
| [数据块2] subfile2.doc (300B有效 + 212B \0填充)               | 512B
├──────────────────────────────────────────────────────────────┤
| [头记录5] 文件: "./subdir/subfile1.txt" type='-', size=200   | 512B
├──────────────────────────────────────────────────────────────┤
| [数据块3] subfile1.txt (200B有效 + 312B \0填充)               | 512B
├──────────────────────────────────────────────────────────────┤
| [头记录6] 文件: "file1.txt"      type='-', size=100          | 512B
├──────────────────────────────────────────────────────────────┤
| [数据块4] file1.txt (100B有效 + 412B \0填充)                  | 512B
└──────────────────────────────────────────────────────────────┘
```

打包文件大小 ≈ 公共头 + 6×512头块 + 4×512数据块 = 公共头 + 5120 字节

6.4 第三步：解包（`CMyPack::unpack`）

调用：`unpack("backup.dbkp", "/home/user/restore")`，目标目录为空。

① 切换工作目录

```
chdir("/home/user/restore");
```

② 依次读取记录并恢复

读取顺序	记录内容	执行操作	结果
1	`"."`, type='D'	`if (filePath_ == ".") continue;`	**跳过**（当前目录已存在）
2	`"file2.log"`, type='-'	`open()` + `write` 50B	在 `/restore/` 下创建 `file2.log`
3	`"./subdir"`, type='D'	`mkdir("./subdir")`	在 `/restore/` 下创建子目录
4	`"./subdir/subfile2.doc"`, type='-'	`open()` + `write` 300B	在 `/restore/subdir/` 下创建
5	`"./subdir/subfile1.txt"`, type='-'	`open()` + `write` 200B	在 `/restore/subdir/` 下创建
6	`"file1.txt"`, type='-'	`open()` + `write` 100B	在 `/restore/` 下创建 `file1.txt`

③ 最终恢复结果

```
/home/user/restore/
├── file1.txt          (100 字节)
├── file2.log          (50 字节)
└── subdir/
    ├── subfile1.txt   (200 字节)
    └── subfile2.doc   (300 字节)
```

✅ **完全还原！** 与源目录结构一模一样。

6.5 关键设计决策总结

设计决策	原因	代码体现
遍历时先记录目录自身	保证解包时父目录先于子文件创建	`result.push_back(path)` 在递归前

设计决策	原因	代码体现
打包/遍历都 `chdir` 到源目录	所有路径变成相对路径，缩短记录长度	`getcwd` + `chdir` + 恢复
数据块固定 512 字节	便于 `lseek` 跳过，对齐读写	`sizeof(data.block)`
用 `lstat()` 而非 `stat()`	防止软链接导致死循环	`lstat(file.c_str(), &buff)`
解包目标目录必须为空	防止覆盖用户现有数据	`unpackCheck` 中遍历检查

7. 代码中的缺陷与改进方向

7.1 ❌ 缺陷1：扩展头记录逻辑错误

在 `packHeaderRecord` 中，计算扩展头数量的逻辑存在问题：

```
int numExtRecordForFile = filePathLen / 200;
int numExtRecordForLink = linkPathLen / 200;
int numExtRecord = max(numExtRecordForFile, numExtRecordForLink);  // ❌ 问题！
```

问题场景：如果文件名需要 1 个扩展头（长度 250），链接路径需要 2 个扩展头（长度 450），则 `numExtRecord = 2`。但循环中：

```
for (int i = 0; i < numExtRecordForFile; i++) {  // i=0, 复制文件名剩余
    // ...
}
for (int i = 0; i < numExtRecordForLink; i++) {  // i=0,1, 复制链接路径剩余
    // ...
}
```

由于 `numExtRecord=2`，两个扩展头都写入了。但**文件名和链接路径的数据会互相覆盖**，因为同一个扩展头的 `filePath` 和 `linkPath` 字段是共存的。

✅ **修复方案**：每个扩展头带类型标识

```
struct ExtendRecord {
    char type;          // 'P' = 文件名扩展, 'L' = 链接名扩展
    char data[200];     // 扩展数据
};

// 打包时
if (filePathLen > 200) {
    ExtendRecord er;
    er.type = 'P';
    memcpy(er.data, file.c_str() + 200, filePathLen - 200);
    sf.writeN(&er, sizeof(er));
}
// 链接路径同理

// 解包时
```

```
while (需要读取扩展头) {
    ExtendRecord er;
    sf.readN(&er, sizeof(er));
    if (er.type == 'P') filePath_ += string(er.data, ...);
    else if (er.type == 'L') linkPath_ += string(er.data, ...);
}
```

7.2 ❌ 缺陷2：目录时间戳恢复不完整

```
// 在 unpackOneByOne 中
if (type_ != 'L') {
    chown(filePath_.c_str(), uid_, gid_);
    utime(filePath_.c_str(), &utb);  // ❌ 目录时间会被后续操作更新
}
```

问题：目录创建后立即恢复时间，但后续向该目录添加文件会更新目录的 `mtime`。

✅ **修复方案**：两阶段恢复

```
第一阶段：只创建所有文件和目录（不恢复元数据）
第二阶段：逆序遍历（从最深子目录到根），恢复每个目录的元数据
```

7.3 ❌ 缺陷3：硬链接未处理

当前代码将硬链接当作普通文件处理，导致：

1. 打包文件膨胀（同一份数据存 N 遍）。
2. 解包后链接关系丢失。

✅ **修复方案**：维护 Inode 映射表

```
// 全局映射表: (设备号, Inode号) → 打包文件中的偏移量
std::map<std::pair<dev_t, ino_t>, off_t> g_InodeMap;

int CMyPack::packOneByOne(...) {
    lstat(file, &buff);
    auto key = std::make_pair(buff.st_dev, buff.st_ino);

    if (buff.st_nlink > 1 && g_InodeMap.find(key) != g_InodeMap.end()) {
        // 这是硬链接！只存引用
        HardLinkRecord hlr;
        hlr.type = 'H';
        hlr.refOffset = g_InodeMap[key];
        sf.writeN(&hlr, sizeof(hlr));
        return 0;
    }

    if (buff.st_nlink > 1) {
        g_InodeMap[key] = sf.getCurrentOffset();  // 记录数据位置
    }
    // ... 正常写入
```

```
}
```

解包时遇到 `'H'` 类型，根据 `refOffset` 读取数据块，然后调用 `link()` 重建硬链接。

7.4 ❌ 缺陷4：符号链接元数据未恢复

对于符号链接，代码跳过了 `chown` 和 `utime`。但符号链接本身也有属主和时间戳。

✅ **修复方案**：使用 `lchown()` 和 `utimensat()`

```
if (type_ == 'L') {
    symlink(linkPath_.c_str(), filePath_.c_str());
    lchown(filePath_.c_str(), uid_, gid_);        // 修改链接自身属主
    utimensat(AT_FDCWD, filePath_.c_str(), times, AT_SYMLINK_NOFOLLOW);  // 修改链接自身时间
}
```

7.5 ⚠️ 缺陷5：`realpath()` 可能绕过排除规则

```
// CFileEnumerator::dirFilter
char absPath[PATH_MAX];
realpath(path.c_str(), absPath);  // ⚠️ 解析软链接到最终目标
```

如果用户想排除 `/tmp/link_to_etc` 这个软链接本身，但 `realpath` 将其解析为 `/etc`，可能绕过排除规则。

✅ **修复方案**：优先使用 `lstat` 判断

```
struct stat buf;
lstat(path.c_str(), &buf);
if (S_ISLNK(buf.st_mode)) {
    // 单独处理软链接排除
}
```

8. 跨平台移植指南（Linux → Windows）

如果学生尝试在 Windows 下实现，需注意以下 API 映射：

Linux 系统调用	Windows 等价 API	注意事项
`lstat()`	`GetFileAttributesExW()`	需判断 `FILE_ATTRIBUTE_REPARSE_POINT`
`readlink()`	`DeviceIoControl(FSCTL_GET_REPARSE_POINT)`	解析 `REPARSE_DATA_BUFFER`
`symlink()`	`CreateSymbolicLinkW()`	**需要管理员权限**（Win10 之前）
`link()`（硬链接）	`CreateHardLinkW()`	NTFS 支持，无需管理员权限
`mkfifo()` / `mknod()`	**不存在**	Windows 无管道/设备文件节点
`opendir()` / `readdir()`	`FindFirstFileW()` / `FindNextFileW()`	路径分隔符用 `\`
`chdir()`	`SetCurrentDirectoryW()`	工作目录切换
`getcwd()`	`GetCurrentDirectoryW()`	获取当前工作目录

Windows 下的四种链接机制

特性	硬链接	目录联接 (Junction)	符号链接	快捷方式
操作对象	仅文件	仅目录	文件/目录	任意
跨卷	❌	✅ (同机器)	✅	✅
需要管理员	❌	❌	✅	❌
本质	同一 MFT 记录	重解析点	重解析点	`.lnk` 文件

Windows 下的路径分隔符处理

```
#ifdef _WIN32
    const char PATH_SEP = '\\';
#else
    const char PATH_SEP = '/';
#endif

// 或使用 C++17 标准库
#include <filesystem>
std::filesystem::path p = "dir";
p /= "subdir";  // 自动使用正确分隔符
```

9. 课堂思考题

Q1：遍历时 `readdir` 的顺序不确定，会影响打包/解包的正确性吗？

答案：不影响，但有前提条件——**递归遍历时必须在进入子目录之前先记录目录自身**（先序遍历）。

- 同层文件的顺序不影响正确性（它们互不依赖）。
- 父目录必须排在子文件之前（否则解包时目录不存在，`open` 子文件失败）。
- 代码中的 `traverseDir` 在递归调用 `traverseDir(subdir)` **之前**先执行了 `result.push_back(path)`，保证了这一点。

Q2：如果源目录中包含指向自身的软链接（`ln -s . self`），遍历会死循环吗？

答案：**不会。**因为 `entry->d_type` 对软链接返回 `DT_LNK`，而不是 `DT_DIR`。代码中只有 `entry->d_type == DT_DIR` 时才递归。即使使用 `lstat` 判断，`S_ISLNK` 也为真，不会进入递归分支。

但如果误用 `stat()`：`stat("self")` 会返回目标目录的信息，`S_ISDIR` 为真，导致递归进入，造成死循环。**这就是必须使用 `lstat()` 的原因！**

Q3：为什么解包时要强制目标目录为空?

答案：防止覆盖用户现有数据。`unpackCheck` 中会遍历目标目录，如果发现任何非 `.` 和 `..` 的条目，直接报错返回 `DESTINATION_DIR_NOT_EMPTY`。

Q4：打包后的文件，数据块为什么要固定 512 字节？为什么不用文件实际大小?

答案:

1. **对齐**：便于使用 `lseek` 快速跳转到任意文件的数据区。
2. **简化读写**：每次读写固定大小，代码简单，减少碎片。
3. **历史原因**：Tar 格式使用 512 字节块（磁盘扇区大小）。
4. **代价**：小文件会浪费空间（如 1 字节的文件占用 512 字节存储）。

Q5：如何修改代码以支持 Windows 下的符号链接?

```
#ifdef _WIN32
#include <windows.h>
// 创建符号链接
CreateSymbolicLinkA(filePath_.c_str(), linkPath_.c_str(),
                    S_ISDIR(mode_) ? SYMBOLIC_LINK_FLAG_DIRECTORY : 0);
#else
symlink(linkPath_.c_str(), filePath_.c_str());
#endif
```

Q6：目录的 `mtime` 为什么不能准确还原？怎么修复?

答案：因为目录创建后，后续向其中添加文件会更新目录的 `mtime`。

修复方案:

1. 第一遍只创建文件和目录，不恢复时间。
2. 记录所有需要恢复元数据的路径。
3. 第二遍**逆序**（从最深层子目录到根目录）调用 `utime()`。

```
// 收集所有需要恢复的路径
std::vector<std::string> dirs;
// 解包时先不调用 utime，只记录

// 最后逆序恢复
for (auto it = dirs.rbegin(); it != dirs.rend(); ++it) {
    utime(it->c_str(), &utb);
}
```

10. 总结

核心知识点

1. **遍历**：`chdir` + 先序递归 + `lstat()` + 路径拼接
2. **打包**：头记录 + 数据块 + 512 字节对齐
3. **解包**：按序读取 + 类型分支 + 元数据恢复
4. **关键原则**：父目录先于子文件、用 `lstat` 不用 `stat`

代码设计亮点

- ✅ 遍历与打包逻辑分离，职责清晰
- ✅ 支持多种文件类型（普通/目录/链接/设备/管道）
- ✅ 使用 `chdir` 简化路径管理
- ✅ 完整的错误检查和资源恢复

可改进方向

- 🔧 扩展头逻辑重构
- 🔧 支持硬链接（Inode 映射表）
- 🔧 目录时间戳后置恢复
- 🔧 符号链接元数据恢复
- 🔧 增量备份与断点续传

> 📖 **推荐阅读**：
>
> - `man lstat`、`man readdir`、`man mkfifo`——理解系统调用
> - ext2 文件系统文档
> - Tar 格式规范（POSIX.1-1988 / UStar）